INGREDION AGENTIC AI FACTORY

Ingredion Agent Workflows

Agent A, the Email Order Intake Agent as primary processor for every email-order case, and Agent B, the Pre-bill Validation and Dispute Prevention Agent, both on Google Cloud per Ingredion's redline v2: the step-by-step flows, the shared runtime they run on, and the engineering review of what remains to settle before Gate 2.

Prepared by Abhishek Nage, Parkar
Date 21 October 2026 · **Version** 1.1 · **Status** for internal review

Basis. Two agents under the Ingredion Agentic AI Factory, drawn step by step from the SOW documents and Ingredion's redline v2 of 20 October 2026: the Email Order Intake Agent (Agent A), now the primary processor for every email-order case Salesforce routes to it, cohort by cohort, and the Pre-bill Validation and Dispute Prevention Agent (Agent B). Both run inside the Ingredion Google Cloud project with Gemini on Vertex AI; Google Cloud services are confirmed in Stage 2.

Contents

One factory, two agents, one Google Cloud project 3

The two agents side by side 5

Agent A · Email Order Intake Agent on Google Cloud 5

Ten steps, where each runs 7

Cohort rollout and the RPA transition 8

Measures: the SOW 2.7 scorecard 8

Production modes 8

Decisions the redline leaves for the stages 9

Operational controls 9

Agent B · Pre-bill Validation and Dispute Prevention Agent on Google Cloud 10

The refined flow 10

Sketch versus refined, element by element 12

Feasibility by component 14

The five rules 15

One schema for both documents 16

Outcomes and who acts 16

What stays open until 23 October 16

What redline v2 changes for Agent B 16

Engineering review · fifteen points to settle in Stages 1 and 2 17

Stages, gates and dates for both agents 20

How this maps onto the repository 21

REVISION OF 21 OCT

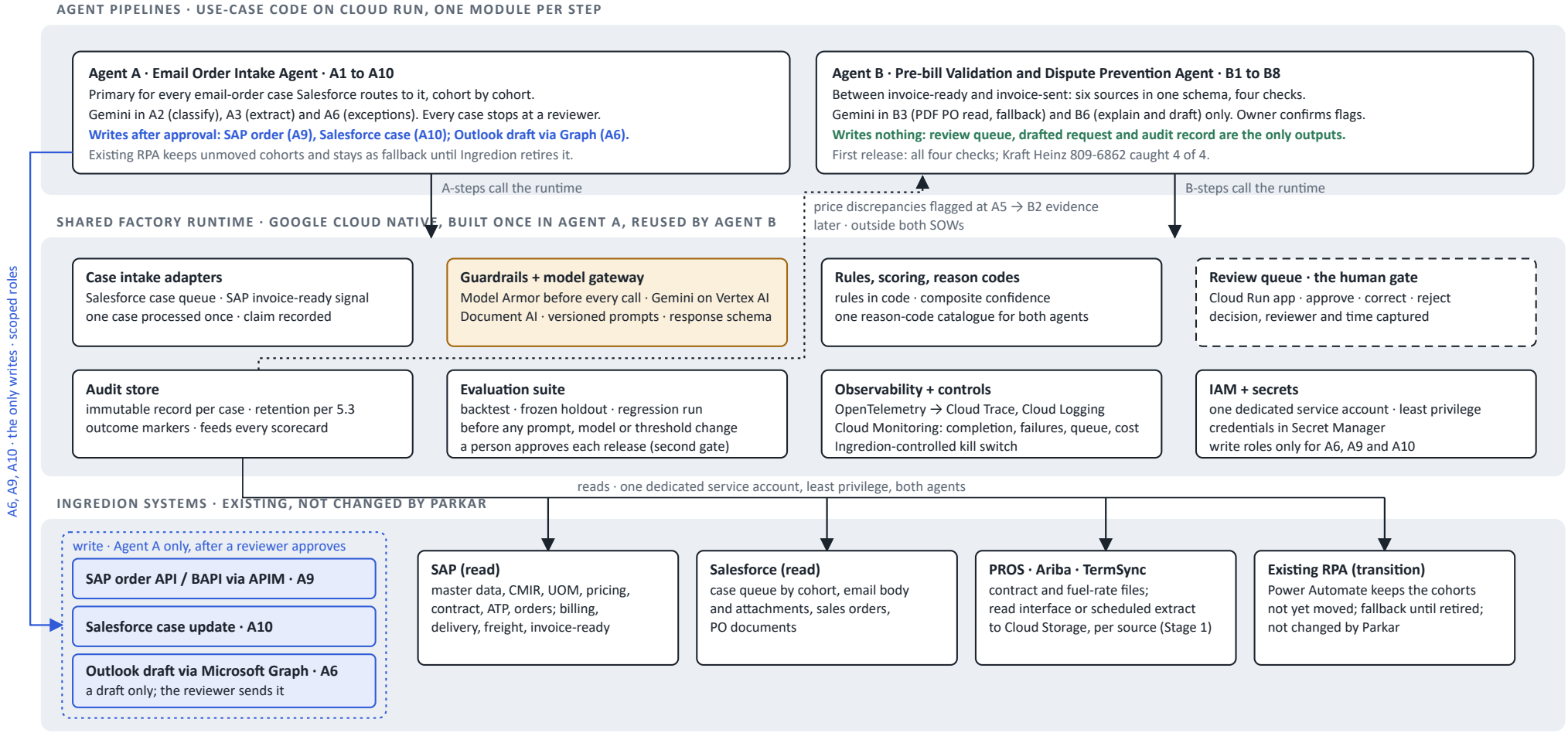
Ingredion's redline v2 changes the platform and the scope of Agent A; the factory split and the eight pre-bill steps stand. What is different from the 19 October page:

1. **Google Cloud, not Azure.** Both agents run on Cloud Run in the Ingredion Google Cloud project (Dev, Test, Prod). The model is an approved Gemini endpoint on Vertex AI; Document AI reads scanned POs; Model Armor screens email and PO text before every model call; Cloud Storage lands scheduled extracts; Cloud Trace, Cloud Logging and Cloud Monitoring carry observability; one dedicated service account, least-privilege IAM and Secret Manager carry access. Services follow the redline and are confirmed in Stage 2.
2. **Agent A processes all email orders, cohort by cohort.** It is no longer a fallback for RPA failures: Ingredion routes each customer cohort to the agent in the Salesforce case queue, the Existing RPA keeps the cohorts not yet moved and stays as fallback until Ingredion retires it, and every exit goes to the manual queue (Customer Service) with the agent's reason on the original case.
3. **Ten steps, generative AI in three.** A6 (exceptions) drafts the customer clarification with Gemini and saves it as an Outlook draft through Microsoft Graph for the reviewer to send; A7 builds the review summary; A9 creates the order through the SAP order API or BAPI via APIM (method approved at Gate 2) and then reads the SAP incompletion log; A10 writes the order number, status, audit note and incompletion follow-ups to the case.
4. **No mapping database.** A4 maps directly to SAP fields (customer, ship-to, bill-to, material, CMIR, UOM). Where validation runs (SAP or Google Cloud) is recommended in Stage 1 and decided at Gate 1.
5. **Every case stops at the reviewer in Phase 1.** Cohort-by-cohort rollout is what keeps that workable; Phase 2 (high-confidence cases skip the gate) is an Ingredion decision at Gate 5.
6. **Agent B: four checks in the first release** (price; freight and surcharge; quantity; payment terms), outcomes Matched · Updated PO required · Fix our side or decide, and an owner per outcome (Customer Service or Commercial; the Ingredion owner, with price changes sitting with CBA, Sales and PROS). The Kraft Heinz anchor must be caught 4 of 4.
7. **Code realigned on 21 October.** Vertex AI, Document AI and Model Armor adapters behind the existing factory interfaces; the Order Intake pipeline rebuilt as A1 to A10 with a primary mode and a fallback mode; 82 tests on stubs and a fake model.

One factory, two agents, one Google Cloud project

What is shared, what is specific, and where the only writes to Ingredion systems happen.

use-case code / shared runtime · generative AI on Vertex AI · writes to an Ingredion system · later connection · layer



The factory split on Google Cloud. Each agent is a set of thin step modules on Cloud Run; the runtime beneath them is identical. Both agents read Ingredion systems through one dedicated service account. Only Agent A writes: the SAP order and the Salesforce case after the human gate, and an Outlook draft the reviewer may send. The dotted link is the one data dependency between the agents: price discrepancies flagged at order entry become evidence for the pre-bill check.

The two agents side by side

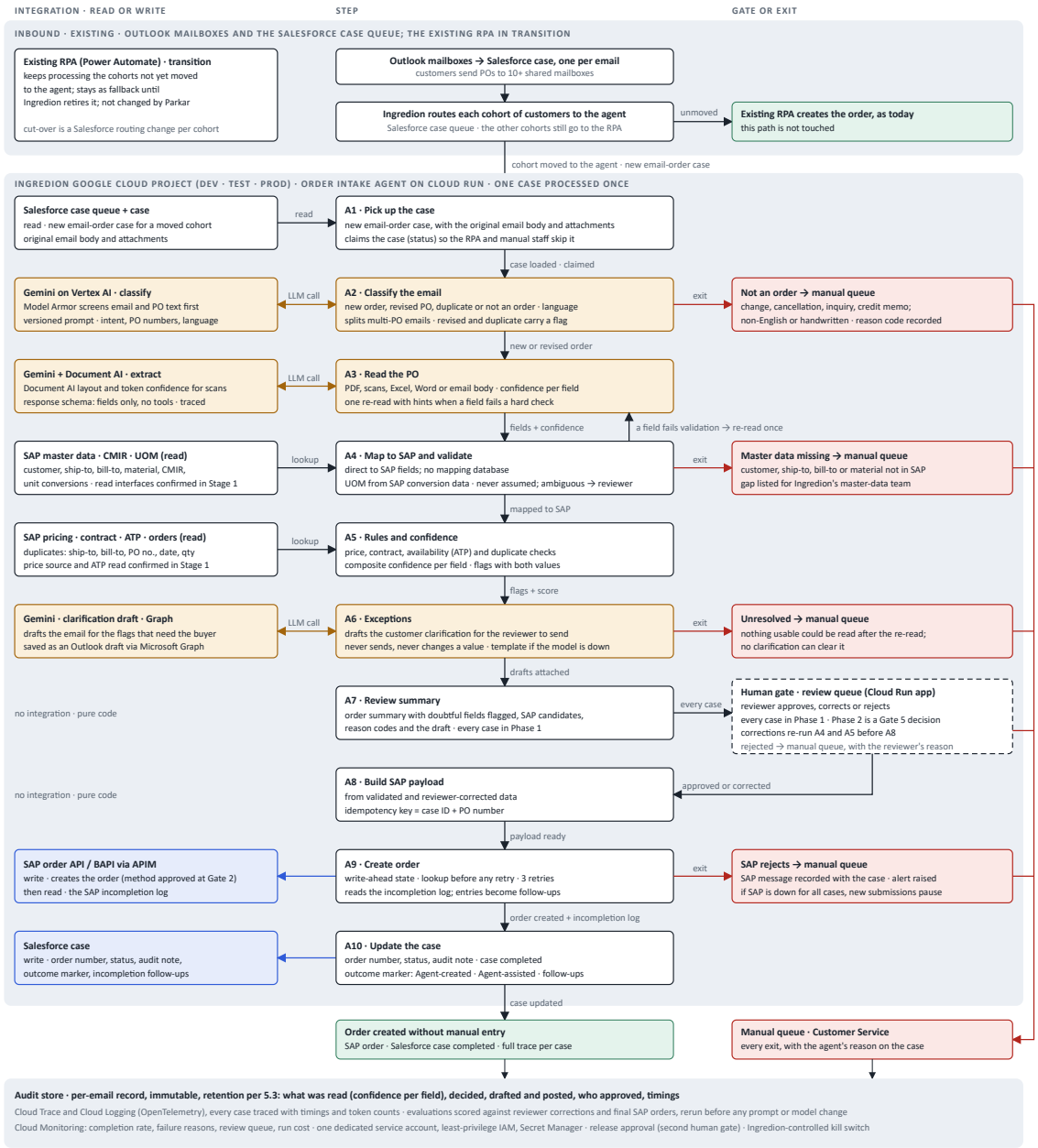
	Agent A · Email Order Intake	Agent B · Pre-bill Validation
Where it sits	At order entry, as the primary processor for the cohorts routed to it; the Existing RPA keeps the rest during the transition	Between invoice-ready and invoice-sent, before the customer portal's three-way match
Trigger	New email-order case in the Salesforce queue for a cohort moved to the agent	Shipped order with a draft invoice ready in SAP
Generative AI (Vertex AI)	A2 classify the email; A3 extract PO fields (with Document AI for scans); A6 draft the customer clarification	B3 read a PDF PO when no structured PO exists (Document AI if scanned); B6 explain the gap and draft the request
Deterministic core	Direct mapping to SAP fields and validation (customer, ship-to, bill-to, material, CMIR, UOM), duplicate check, price, contract and ATP rules, composite confidence, payload build	One canonical schema, line pairing, four checks: price, freight and surcharge, quantity, payment terms
Reads	Salesforce case queue and cases; SAP master data, CMIR, pricing, contract, ATP, orders	SAP, PROS, Salesforce, Ariba, TermSync, contract and fuel-rate files (read interface or scheduled extract to Cloud Storage)
Writes	**SAP order (A9), Salesforce case (A10)** after reviewer approval; an Outlook draft via Graph (A6) that only the reviewer sends	**None**; owners act outside the agent
Human gate	Every case in Phase 1; Phase 2 (high-confidence cases skip the gate) is an Ingredion decision at Gate 5	Every case that is not Matched; matched invoices are recorded only
Outcomes	Agent-created · Agent-assisted · Returned to the manual queue (with reason code)	Matched · Updated PO required · Fix our side or decide
Exits	A2 not an order; A4 master data missing; A6 unresolved; gate reject; A9 SAP rejects → manual queue (Customer Service)	No exits; a recheck from B2 runs when the updated PO arrives
Measures	SOW 2.7 scorecard: completion rate (judged against the Existing RPA's automation rate on the same cohorts), field-level accuracy, undetected errors, reviewer confirmation, handling time and run cost; thresholds set at Gate 1 from the 500-PO backtest	Catch rate ≥ 95% incl. Kraft Heinz 4/4; false alarms ≤ 10%; traceability 100%; reviewer agreement ≥ 85%; time saved ≥ 30%; zero unattended actions
Platform	Cloud Run · Vertex AI (approved Gemini endpoint) · Document AI · Model Armor · Cloud Storage · audit store · Cloud Trace, Logging, Monitoring · IAM and Secret Manager; Dev, Test and Prod in the Ingredion Google Cloud project; services confirmed in Stage 2	
Status	Statement of Work No. 07, Ingredion redline v2 (20 Oct 2026): the contract text for both agents; commences 26 Oct 2026; scorecard thresholds set at Gate 1 from the backtest; scope and fees not reopened at Gate 1	Same SOW; step list confirmed in Stage 2; variances confirmed at Gate 1; invoice-ready definition agreed in Stage 2

AGENT A

Email Order Intake Agent on Google Cloud

Processes all email orders from Salesforce, cohort by cohort. Ten steps, generative AI in three (A2, A3, A6), one human judgment gate, five exits to the manual queue with the agent's reason on the original case.

deterministic code · generative AI on Vertex AI · write to an Ingredion system · exit to the manual queue · people · layer



Where each step runs and what it touches, on Google Cloud. Gemini is called three times at most: to classify, to extract, and to draft the clarification when a flag needs the customer. Everything that maps, validates, scores, builds the payload and writes is ordinary code. Five exits lead to the manual queue, with a reason code on the original Salesforce case. Writes happen in three places only: the SAP order and the Salesforce case after the reviewer, and an Outlook draft the reviewer may send.

Ten steps, where each runs

Step	Runs on	Reads · writes	Exit, loop or gate	Reason codes (examples; catalogue agreed in Stage 2)
A1 Pick up the case	Cloud Run, scheduled	Reads the Salesforce case queue for the moved cohorts and the case (email body, attachments). Writes a claim status on the case	None	—
A2 Classify the email	Gemini behind Model Armor	Reads email text and attachment text	Exit when not an order or when the format is out of scope; revised PO and duplicate submissions continue with a flag; multi-PO emails are split	`NOT_ORDER.CANCELLATION`, `NOT_ORDER.INQUIRY`, `FORMAT.NON_ENGLISH`, `EMAIL.REVISED_PO`, `EMAIL.DUPLICATE_SUBMISSION`, `EMAIL.MULTI_PO`
A3 Read the PO	Gemini + Document AI	Reads PDF, scans, Excel, Word, email body. Returns fields with confidence in a fixed response schema	Loop: a field that fails a hard check at A4 is re-read once with hints	`EXTRACT.LOW_CONFIDENCE`, `EXTRACT.LINE_TOTALS_MISMATCH`, `EXTRACT.NO_LINES`
A4 Map to SAP and validate	Cloud Run code	Reads SAP master data: customer, ship-to, bill-to, material, CMIR, unit conversions. No mapping database	Exit when a partner or material is not in SAP; ambiguous matches and units go to the reviewer with candidates	`MASTER.SHIPTO_NOT_FOUND`, `MASTER.MATERIAL_NOT_FOUND`, `CMIR.MULTIPLE_MATCHES`, `UOM.AMBIGUOUS`
A5 Rules and confidence	Cloud Run code	Reads SAP pricing, contract, ATP and existing orders	Flags only; the case continues to A6 and A7	`PRICE.PO_VS_SAP`, `CONTRACT.NONE`, `ATP.SHORT`, `DUP.POSSIBLE`
A6 Exceptions	Gemini behind Model Armor; Microsoft Graph	**Writes** an Outlook draft in the shared mailbox (never sends)	Exit when nothing usable could be read and no clarification can clear it	`EXCEPTION.UNRESOLVED`, `FORMAT.UNREADABLE`
A7 Review summary	Cloud Run code	No integration	Gate: every case in Phase 1; in Phase 2 a clean, high-confidence case continues	—
Gate	Reviewer in the review queue app	Records approve, correct or reject with reviewer and time	Approved or corrected → A8 (A4 and A5 re-run on corrected values). Rejected → manual queue	`REVIEW.REJECTED`, `REVIEW.CORRECTED`, `REVIEW.RECHECK_FLAGGED`

Step	Runs on	Reads · writes	Exit, loop or gate	Reason codes (examples; catalogue agreed in Stage 2)
A8 Build SAP payload	Cloud Run code	No integration	Unresolved partner or material → back to the reviewer	—
A9 Create order	Cloud Run code → SAP order API / BAPI via APIM	**Writes** the order; writes a "submitting" state before the call; looks the order up by its reference before any retry; then reads the incompletion log	Exit when SAP rejects, or after 3 retries when SAP is unavailable; pause when SAP is down for all cases	`SAP.REJECTED`, `SAP.UNAVAILABLE`, `SAP.RECONCILED`, `SAP.INCOMPLETE`
A10 Update the case	Cloud Run code → Salesforce	**Writes** order number, status, audit note, the outcome marker and the incompletion follow-ups; retried independently of A9	None	Outcome: `AGENT_CREATED`, `AGENT_ASSISTED`, `RETURNED`

Cohort rollout and the RPA transition

- **Cut-over is a routing change, not a code change.** Ingredion assigns a cohort of customers to the agent's Salesforce queue; from then on the agent is the primary processor for those customers and the Existing RPA no longer sees them. Moving a cohort back is the same change in reverse, which is also how the kill switch fails back.
- **Shadow first, per cohort.** A cohort runs in shadow (the agent processes, the RPA still creates the orders, results are compared) before it is moved, so the agent's completion rate is known against the RPA's on the same customers.
- **The RPA stays deployable** until Ingredion retires it (an Ingredion decision informed by the closure report). Parkar does not change it.
- **Volume.** Primary processing for all cohorts is several times the failed-case volume the earlier, failure-only draft was sized for; reviewer capacity, Gemini quota and run cost per cohort are measured in shadow mode before each cut-over.

Measures: the SOW 2.7 scorecard

SOW No. 07 (section 2.7) sets six measures for Agent A: completion rate (share of email order cases completed as SAP sales orders), field-level accuracy, undetected errors, reviewer confirmation, handling time and run cost. Their thresholds are set at Gate 1 from the Stage 1 backtest of 500 past POs including RPA-failed POs, with the final SAP orders as the answer key, and they are the basis for Gates 4 and 5. The earlier targets in the clarification responses (recovery ≥ 50% at Gate 1, ≥ 70% at Gate 4, ≥ 80% before Phase 2; undetected errors ≤ 2% of key fields, ≤ 0.5% before Phase 2) were written for the failed-case scope and now serve only as Parkar's starting proposal; the headline measure we propose is the **completion rate on a cohort against the Existing RPA's automation rate on the same cohort**. From Stage 5 Parkar reports twelve measures from the audit trail and dashboards (cases received and completed, orders created, incompletion items and follow-ups, reviewer corrections, cases returned to the rework path, failure reasons by category, accuracy and undetected errors, handling time, run cost, trends against the baseline).

Production modes

- **Shadow (per cohort):** live cases, no SAP order, no Salesforce update, no Outlook draft; results compared with the RPA's or the manual order; tuning.

- **Assisted (Phase 1):** every result confirmed or corrected by an Ingredion reviewer before A8; corrections feed the evaluation set.
- **Phase 2 (Gate 5 decision):** high-confidence cases skip the gate; anything with a price, contract or availability discrepancy, a rule failure, a revised or duplicate submission, or a possible SAP duplicate always goes to the reviewer.

Decisions the redline leaves for the stages

- Google Cloud services and the review queue app design `[Stage 2]`; the SAP method, order API or BAPI, through APIM `[Gate 2]`.
- Validation location, SAP or Google Cloud `[recommended Stage 1, decided Gate 1]`.
- Read interface or scheduled extract per source; SAP pricing, contract and ATP reads `[Stage 1]`.
- Cohort order and shadow duration per cohort; reviewer capacity per cohort `[Gate 1]`.
- Phase 2 criteria for skipping the gate `[Gate 5]`.

Operational controls

Control	What the design commits to	Engineering note
One case, once	The agent claims each case in Salesforce; the RPA and manual staff skip claimed cases; unique reference = case ID + PO number	A persisted per-case state machine (picked up → classified → extracted → validated → scored → exceptions → in review → approved → submitting → created → case updated, or returned) is the idempotency record.
Duplicate orders	Revised and duplicate submissions flagged at A2; duplicate-order check against SAP at A5; lookup by reference before any retry at A9	The pre-retry lookup needs the SAP order field that carries the customer PO number; confirmed in Stage 1.
SAP timeout	Look the order up, then retry up to 3 times with increasing waits, then exit to the manual queue and alert; pause all submissions if SAP is down	Pause automatically after N consecutive failures; resume by hand; both logged.
Incompletion log	Read after every creation; entries become follow-ups on the Salesforce case and a flag on the audit record	The log is advisory: an order exists even if the read fails; the failure is recorded, never retried as a creation.
Outlook drafts	A6 saves the clarification as a draft in the shared mailbox through Microsoft Graph; only the reviewer sends it	Graph permission scoped to drafts on the shared mailbox (no send permission); the draft text also sits on the review item, so a Graph failure never blocks the case.
Kill switch	Ingredion-controlled; stops new cases immediately; in-flight cases complete or return; each use logged	Failing back is the same Salesforce routing change that moved the cohort; the RPA keeps running for unmoved cohorts throughout.
Guardrails	Model Armor screens email and PO text before every model call; the model answers in a fixed response schema with no tools	Model Armor fails closed; a blocked text parks the case for a reviewer and never reaches Gemini.
Learning	Corrections captured and added to the evaluation set; prompts, rules and thresholds adjusted; every change re-tested and approved by a person; no self-learning	Holdout frozen for Stage 4; evaluation runs on every build (Gate 3 criterion).
Data	PO data stays in the Ingredion Google Cloud project; retention per 5.3; no masking in non-production	Region, data residency and VPC Service Controls perimeter are Stage 2 confirmations.

AGENT B

Pre-bill Validation and Dispute Prevention Agent on Google Cloud

Unchanged in its eight steps since 16 October and moved to the Ingredion Google Cloud project by redline v2, with all four checks in the first release. The verdict on the whiteboard sketch, the refined flow and the rules are kept below; what redline v2 changes for this agent follows at the end of the section.

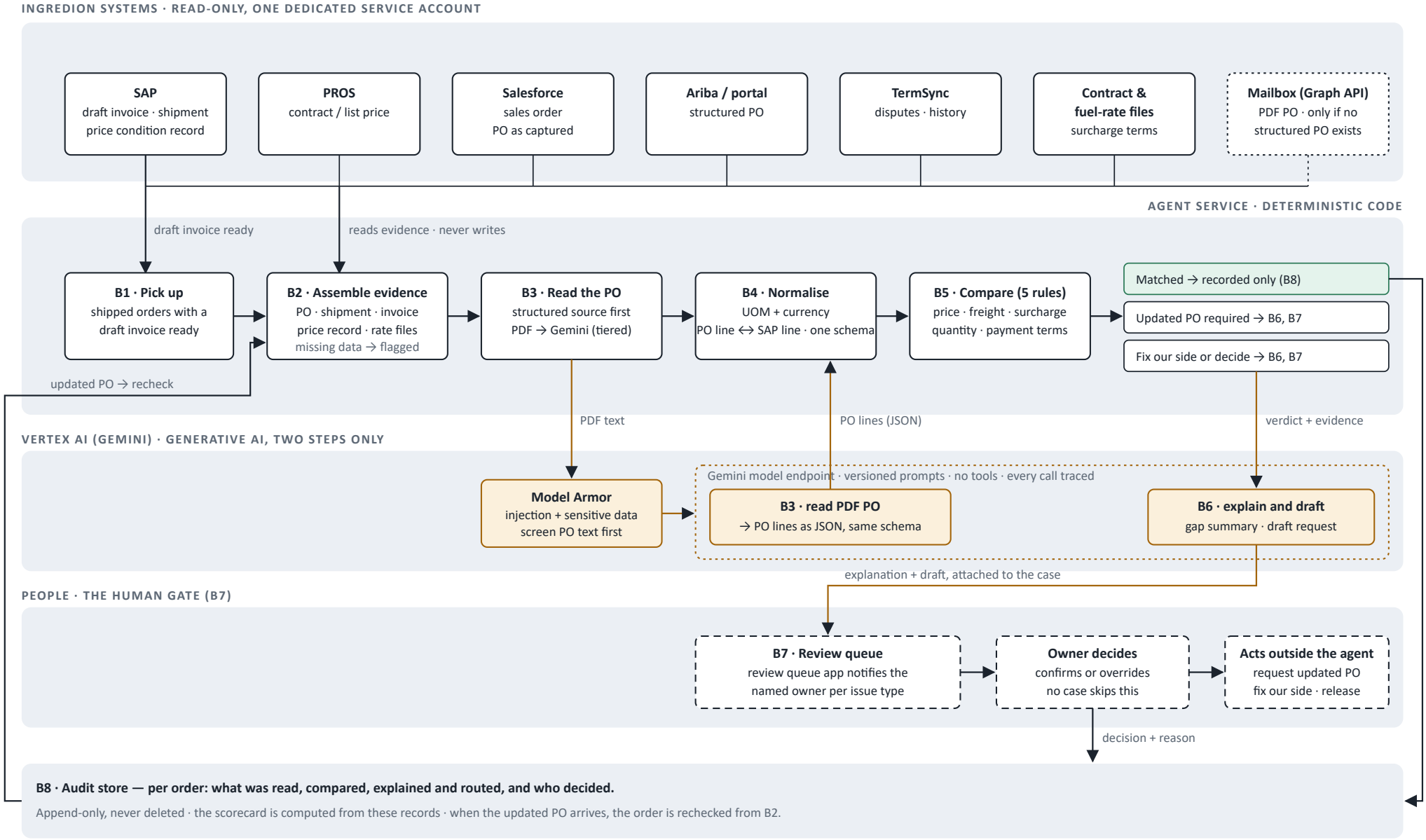
FEASIBLE

The skeleton in the sketch is the SOW's own design: pull PO and invoice into one schema, let rules decide, invoke a Gemini agent only afterwards to write the reasoning, and notify people through a review queue app. Six things change between the sketch and what we will build:

1. **Code owns the verdict, the model owns the explanation.** The box labelled "Draft Reasoning Verdict" splits in two: B5 (rules) produces the outcome; B6 (Vertex AI) only explains it and drafts the request. The model can never change an outcome.
2. **One verdict becomes three outcomes.** Matched · Updated PO required · Fix our side or decide. A matched invoice is recorded and never waits for a person or a model call.
3. **"Ingredion Platform" is six named read-only sources**, reached through one dedicated identity that can read and never write: SAP, PROS, Salesforce, Ariba, TermSync, contract and fuel-rate files.
4. **The email → Graph API → PDF path is a fallback, not the main road.** B3 reads structured PO data first; a PDF is parsed only when no structured PO exists, and its text is screened by Model Armor before any model sees it.
5. **The review queue app records the decision**, not just an email. The named owner confirms or overrides every flagged case (B7); nothing skips this step.
6. **Four pieces the sketch did not have:** the B1 trigger from SAP, missing-data handling in B2, the append-only audit trail (B8) with the recheck loop when the updated PO arrives, and the evaluation scorecard that gates every release.

The refined flow





Where each step runs. Deterministic code picks up the invoice, assembles the evidence, normalises both documents into one schema and decides. Gemini on Vertex AI is called in two places only: to read a PDF PO when nothing structured exists, and to explain a gap and draft the request after code has decided. A person confirms every flagged case. Matched invoices are recorded and never wait for a model or a reviewer.

Sketch versus refined, element by element

In the sketch	Refined	Why (source)
Connection · auth client (router + controller)	Kept. Same service shape as the DOIT Vertex AI integration: a thin API, a controller per step, a Vertex AI gateway. Secrets move to a dedicated service account and Secret Manager; the identity that reads client systems is read-only.	SOW: access and secrets; "writes to no Ingredion system".
"Ingredion Platform" → PO and invoice	Six named read-only sources. SAP supplies the draft invoice, shipment and price condition record; PROS the contract price; Salesforce the order and the captured PO; Ariba the structured PO; TermSync dispute history; the contract and fuel-rate files the surcharge terms.	SOW B2 and target architecture. Ariba and TermSync read access is a Stage 1 assumption.

In the sketch	Refined	Why (source)
Email → Graph API → PO (pdf, any form) → parse	Demoted to a fallback inside B3. Structured PO data is read first; the PDF is parsed only when no structured PO exists, its text screened by Model Armor, Document AI used only for scans. Mailbox scanning per invoice is unreliable for matching an email to an order; prefer the PO attachment stored with the sales order.	SOW B3 tiered read; the mailbox path is the Order Intake use case's job.
Relational / JSON · "data compared follows same schema"	Kept and made explicit as B4: one canonical schema for PO and invoice, with unit-of-measure and currency conversion and line pairing done before any rule runs.	SOW B4; business data-trust table (e.g. BAG→KG, billing-date currency).
Comparison based on rules (5 checks) → verdict	Kept as B5. The five checks are the business briefing's rows (price, freight, fuel surcharge, quantity, payment terms); the SOW groups freight and surcharge. First release switches on price and freight/surcharge; quantity and terms follow. One verdict becomes three outcomes.	SOW B5 and rollout table; briefing slide 9 and 10.

In the sketch	Refined	Why (source)
Agent invoke → Vertex AI (sys prompt, logic, tools) → review queue app → "Draft Reasoning Verdict"	Split. B6 on Vertex AI writes the explanation per gap and the draft PO-update request or fix note; it never produces or alters the verdict. Tools are read-only (case evidence). The review queue app is the B7 queue, where the owner's confirm or override is captured. Matched invoices skip B6 and B7.	SOW B6, B7: "no case skips this"; "rules decide, a person acts".
Not drawn	B1 trigger from SAP's shipped-with-draft-invoice list; missing-data outcome in B2; B8 append-only audit trail; recheck from B2 when the updated PO arrives; evaluation scorecard and release approval; Dev, Test and Prod in the Ingredion Google Cloud project.	SOW B1, B2, B8, observability and governance, environments.

Feasibility by component

Component	Verdict	What it takes
SAP read: draft invoices, shipment, price condition	**Feasible**	Read-only service identity on the billing due list and pricing conditions. Access tickets run about five working days; agreed extracts keep Stage 1 moving if direct access is late.
PROS, Salesforce, Ariba, TermSync, rate files	**Feasible, assumed**	Ariba and TermSync read access is an assumption to confirm in Stage 1. Each source gets a connector behind a read-only interface with a stub for Dev.

Component	Verdict	What it takes
Graph API mailbox read for PDF POs	**Feasible, narrow use**	App registration with mail read scoped to specific shared mailboxes. Matching a message to an order is the weak point, so use it only when the PO exists nowhere else.
PDF PO extraction on Vertex AI (+ Document AI for scans)	**Feasible**	Parkar's invoice agent reached about 93% straight-through on mixed formats. Output must land in the canonical schema with per-field confidence; low confidence routes to "missing data", never to "matched".
Rules engine, five checks, one schema	**Feasible, and the core**	Pure code, unit-tested, no model. Needs UOM tables (lb ↔ CWT, BAG → KG), currency rates by billing date, PO tolerances. Variances and owners are confirmed on 23 Oct.
Gemini agent for explanation and draft	**Feasible**	Versioned prompts; Model Armor in front; tools read-only; the evaluation set is rerun before any prompt or model change and a person approves the release.
review queue app notification	**Feasible, channel TBC**	The SOW leaves the B7 channel open. Whatever is chosen must capture the owner's decision back into the case: an adaptive card, a small queue page, or structured reply parsing.
Audit trail and recheck loop	**Feasible**	Append-only store with soft updates and a per-order record; a rerun from B2 is triggered when the updated PO lands.
"Writes to no Ingredion system"	**Design constraint**	Enforced by the identity's roles, not by convention: the only writable stores are the agent's own queue and audit records.
Observability and scorecard	**Feasible**	OpenTelemetry into Cloud Trace with OpenTelemetry; the scorecard job reads the audit records; Cloud Monitoring alerts on catch rate, false alarms, missing data, queue depth and cost.

The five rules

Check	Rule	Release
Unit price	Exact after UOM and currency conversion; SAP price at billing date versus PO price	1
Freight	Any charge not on the PO is flagged	1
Fuel surcharge	Any charge not on the PO is flagged, even if the contract allows it	1
Quantity	Shipped quantity inside the PO range after UOM conversion; no tolerance stated means exact; over-shipments checked first	2
Payment terms	Exact match; a backstop while the root fix lands at contract and order setup	2

Anchor case for the test set: Kraft Heinz sales order 809-6862, which fails all five (40.14 vs 47.00 USD/CWT; a 300 USD surcharge not on the PO; 170,250 lb shipped against 170,000; 110 against 120 days). Clean, single-error, multi-error, missing-data and borderline orders sit beside it so the rules are not tuned to one order.

One schema for both documents

PO and invoice are mapped into the same shape before B5 runs, so every rule compares like with like and every number carries the record it came from.

```
{
  "doc_type": "po" | "invoice",
  "order_ref": "809-6862",
  "source": "ariba" | "sap" | "salesforce" | "pdf",
  "currency": "USD",
  "payment_terms_days": 120,
  "lines": [{
    "line_no": 1, "material": "…",
    "qty": 170000, "qty_uom": "LB",
    "tolerance_pct": null,
    "unit_price": 40.14, "price_uom": "CWT",
    "price_date": "2026-08-14"
  }],
  "charges": [{ "type": "fuel_surcharge", "amount": 300.00 }],
  "evidence": [{ "field": "unit_price",
                 "system": "SAP", "record": "condition A005/…" }]
}
```

Outcomes and who acts

Outcome	When	What happens next	Owner (proposed)
Matched	All five checks pass and no data is missing	Recorded in B8; the invoice goes out as today. No model call, no reviewer.	Nobody
Updated PO required	SAP is right and the PO is stale (valid price change, added freight or surcharge, quantity, terms)	B6 drafts the request with evidence; B7 owner confirms; the owner asks the customer for the updated PO; recheck from B2 when it arrives.	Pricing & Billing for price and surcharge; Customer Service for quantity; AR & Collections for terms
Fix our side or decide	SAP is wrong, a source is unclear, or evidence is missing (including a missing SAP price)	B6 drafts an internal note; B7 owner decides; never treated as matched. Missing prices are flagged to the pricing teams, not fixed here.	Pricing & Billing, or PoC lead and IT data owners for missing data

What stays open until 23 October

- Named owner per issue type, and the mailbox or channel for B7 notifications.
- Allowed variances per check, and "which source wins" sign-off.
- Read access to Ariba and TermSync; the Google Cloud services list; Dev project from Stage 1.
- How the updated-PO request is run today (who, channel, customer response time) and the fallback when it is late.
- Finance reviewer for the value method; baseline minutes per case for the time-saved measure.

What redline v2 changes for Agent B

- **Platform.** B1, B2, B4, B5 and B8 run as agent code on Cloud Run in the Ingredion Google Cloud project; Gemini on Vertex AI serves B3 and B6; Document AI is optional for scanned POs (confirmed in Stage 2); Model Armor screens PO text before every model call; scheduled extracts for sources with no read interface land in Cloud Storage; the audit store is immutable with retention per 5.3; Cloud Trace, Logging and Monitoring carry the dashboards for

catch rate, false alarms, missing data, queue and cost; one dedicated service account with read-only, least-privilege access to all six sources.

- **Four checks in the first release**, not a staged rollout: price exact after UOM and currency conversion; freight and surcharge, any charge not on the PO flagged; quantity within the PO range, over-shipments first; payment terms exact. The Kraft Heinz anchor must be caught 4 of 4.
- **Outcome names and owners.** Matched (nobody acts; the result is recorded) · Updated PO required (Customer Service or Commercial requests the updated PO from the customer, using the agent's draft; the order is rechecked when it arrives) · Fix our side or decide (the Ingredion owner corrects Ingredion's side, with price changes sitting with CBA, Sales and PROS, or a person decides).
- **What the agent never does:** no change to SAP or PROS, no holding or releasing invoices, no message to a customer.
- **Still to confirm:** read access to Ariba and TermSync (Stage 1); read interface or scheduled extract per source (Stage 1); the step list (Stage 2); the agreed variances (Gate 1).

Engineering review · fifteen points to settle in Stages 1 and 2

Updated for redline v2 and the redlined SOW No. 07 (20 October 2026). Points settled by the redline (validation source, review queue location, scope wording for the fallback design) are closed; the platform move, primary processing and the SOW's new sections (5.3, 5.4, 1.5, 1.7) add new ones.

#	Finding	What the documents say	Risk if left	Recommendation	Settle at
1	**Scope and measures are now contractual; the sample must match them**	SOW No. 07 (2.1, 2.4, 2.7) makes Agent A the primary processor for all email orders, with a scorecard (completion rate, field-level accuracy, undetected errors, reviewer confirmation, handling time, run cost) whose thresholds are set at Gate 1 from a backtest of 500 past POs "including RPA-failed POs" (8.2(b)); scope and fees are not reopened at Gate 1 (8.2(c)).	A sample weighted to failures sets thresholds for the wrong population; a business case built on it does not survive assisted mode.	Agree the sample composition in week 0 (RPA successes and failures per cohort, drawn from the June to July 2026 baseline period); completion rate per cohort against the Existing RPA's automation rate on the same cohort as the headline measure; undetected errors measured against the final SAP orders and, from assisted mode, against post-creation change history (requirements item 28).	Week 0; Gate 1
2	**Cross-cloud connectivity is on the critical path**	The agent runs in the Google Cloud project; the SAP order API sits behind Azure API Management, Salesforce is SaaS, Outlook drafts go through Microsoft Graph.	Every integration crosses a cloud boundary; firewall, private connectivity and allow-listing take weeks and are owned by three parties.	Decide in week 0 between private connectivity (Interconnect or VPN to the Azure network) and allow-listed egress with mTLS or OAuth to APIM; register the Graph application and the Salesforce connected app in week 0.	Week 0; Stage 2 A-D11
3	**Controlled generation on Gemini needs a tested schema**	A2, A3 and A6 return fields in a fixed schema.	Gemini's schema dialect rejects parts of a Pydantic JSON schema; an untested schema fails at runtime or silently loses fields.	Translate the schema (done in the gateway), fall back to JSON mode when rejected, validate every answer with the Pydantic model, and run the backtest with controlled generation on to confirm it holds on real POs.	Stage 1 A-D5

#	Finding	What the documents say	Risk if left	Recommendation	Settle at
4	**Document AI or Gemini-native reading, per format**	A3 uses Gemini with Document AI "where needed" for scans.	Reading scans with Gemini alone loses the token confidence the composite score relies on; running Document AI on every digital PDF costs time and money for nothing.	Document AI for scans and images, Gemini directly for digital PDFs, Excel and Word through readers; decide the routing rule on the Stage 1 sample and record the mean confidence per route.	Stage 1 A-D2, A-D3
5	**Model Armor template is Ingredion's, and it fails closed**	Model Armor screens email and PO text before every model call.	A template that is too strict parks real POs for review; one that is too loose lets document text steer the model. Neither is visible until live traffic.	Ingredion owns the template; measure its block rate on the backtest and in shadow mode; a blocked text parks the case, never calls the model, and is counted on the dashboard.	Stage 2 A-D9; shadow report
6	**Reviewer load at full volume**	Every case stops at the reviewer in Phase 1; Phase 2 is a Gate 5 decision.	Primary processing of all cohorts is several times the failed-case volume; a queue nobody can clear becomes the new manual backlog.	Size reviewer capacity per cohort before each cut-over; shadow each cohort first; put the review app on Cloud Run with single sign-on; define the Phase 2 straight-through criteria early so Gate 5 is a decision, not a design.	Gate 1; Gate 5
7	**Failing back to the RPA must be a routine operation**	The Existing RPA keeps unmoved cohorts and stays as fallback until Ingredion retires it; Parkar does not change it.	If the RPA is left to decay while cohorts move, there is no fallback when it is needed.	Keep the RPA deployable for every cohort until retirement; the kill switch fails back by the same Salesforce routing change that moved the cohort; rehearse once per cohort in shadow; Accenture named as the RPA owner in the RACI.	Gate 2; runbook A-D25
8	**Exactly-once depends on the SAP method**	A9 creates the order through the SAP order API or a BAPI via APIM; the method is approved at Gate 2.	A BAPI call has no idempotency key; a timeout after creation leaves no reference to look up, and a retry creates a second order.	Whatever the method, the lookup before any retry needs the order field that carries the customer PO number and sold-to; APIM can hold the idempotency key and reject a repeat; the write-ahead state and reconciliation on restart stay.	Gate 2; Stage 2 A-D10
9	**Graph permissions for drafts only**	A6 saves the clarification as an Outlook draft; the reviewer sends it.	A broad mail permission turns the draft into a message the agent could send, which the design forbids.	Application permission scoped to the shared mailbox, read and write drafts only, no send; the draft carries the case reference; the draft text also sits on the review item so Graph is never a blocker.	Stage 2 A-D11

#	Finding	What the documents say	Risk if left	Recommendation	Settle at
10	**Data residency and perimeter on Google Cloud**	PO data never leaves the Ingredion project; retention per 5.3; the AI-use approval was written for Azure.	Regions, Vertex AI data-use terms, VPC Service Controls and customer-managed keys are Ingredion security decisions; without them the prototype cannot touch real POs.	Re-issue the data approval naming Vertex AI, the region and the models; decide VPC Service Controls and CMEK in Stage 2; Document AI and Model Armor run in the same region.	Week 0; Stage 2
11	**Quota and run cost at primary volume**	Cloud Monitoring tracks completion rate, failure reasons, review queue and run cost.	Gemini tokens per minute, Document AI pages and Cloud Run concurrency are quotas, not elastic; a cohort cut-over can hit them on day one.	Measure tokens, pages and latency per case in shadow for each cohort; request quota ahead of cut-over; budget alerts per environment.	Shadow report per cohort
12	**Computed confidence and an honest answer key**	A3 returns a confidence per field; the backtest scores against final SAP orders.	Self-reported confidence is uncalibrated, and final orders contain corrections not visible in the PO; both inflate or deflate the Gate 1 numbers.	Composite per-field score from Document AI confidence, layout agreement, arithmetic consistency and master-data match, calibrated on the backtest; classify every mismatch as document-recoverable or not and report both shares.	Stage 1 A-D5; Gate 1
13	**Security and production-readiness testing is contractual (5.4)**	Parkar must run static analysis against an agreed secure-coding standard, secrets scanning, dependency scanning on every build, authorization tests of the service identity, adversarial prompt-injection tests before each release and a rehearsed rollback in Test before Gate 4; critical and high findings close before Gate 4, medium before Gate 5.	A pipeline without these from the first Stage 3 build means a scramble before Gate 4; prompt-injection tests written late test guardrails nobody tuned.	Put static analysis, secrets and dependency scanning in the pipeline on day one of Stage 3 (requirements item 26 asks Ingredion for the standard, the tools and the severity scale); keep a prompt-injection corpus in the repository and run it against Model Armor and the pattern guard in the evaluation suite; script the rollback (previous prompts, models, rules, configuration) and rehearse it in Test.	Stage 2 A-D12; Stage 3; Gate 4
14	**Pre-existing materials and the repository (1.7, 5.1)**	All Agent Work Product is Ingredion's and lives only in Ingredion's repository; Parkar must identify in writing any pre-existing materials before incorporating them and may use them only with Ingredion's approval; no copies are retained after closure.	The shared factory runtime in `Ingredion/` is exactly such material; built on silently, it becomes a dispute at Gate 6 and a licence question for the AMS partner.	Declare the factory runtime and its open-source dependencies in writing before Stage 3 and list them in A-D16a; move the code into Ingredion's repositories in week 0 and develop there; keep the generic parts separable so Parkar can reuse them elsewhere without Ingredion data or configuration.	Week 0; before Stage 3

#	Finding	What the documents say	Risk if left	Recommendation	Settle at
15	**Run model and Knowledge Transition to the AMS partner (1.5, Stage 6)**	Before Gate 4 the release readiness pack must carry a run model agreed with Ingredion (operating owner, monitoring and alert routing, severity definitions and escalation, hand-off to warranty and to the AMS partner, indicative run cost); Stage 6 adds reverse shadowing in which the AMS partner operates the agent with Parkar support, signed off at Gate 6; a 90-day warranty follows.	Without a named run owner and AMS partner early, the run model is written for nobody and Gate 6 has no one to accept the handover.	Ask for the run owner and the AMS partner by Gate 4 (requirements item 27); design the runbook, dashboards and alert routing for that operator from Stage 2; carry the incident severities (Sev 1 response 2 business hours, Sev 2 four hours, Sev 3 one business day) into the health and kill-switch design of the ops router.	Gate 4; end of Stage 5; Gate 6

Stages, gates and dates for both agents

Stage	Agent A · Email Order Intake (SOW No. 07, redline v2)	Agent B · Pre-bill Validation (SOW No. 07, redline v2)	Gate decided by
Week 0 mobilisation	Week of 26 October 2026 · Google Cloud Dev project, service accounts, connectivity decision, Graph and Salesforce apps, the 500-PO historical sample within 5 working days	Week of 26 October 2026 · access to the six sources and the sample of past orders within 5 working days	—
1 Discovery and Assessment	Weeks 1 to 3 · baseline per cohort, exception map, architecture, prototype and backtest, validation-location recommendation · Gate 1 at week 3	Weeks 1 to 3 · baseline, test cases, gap map, architecture, prototype and backtest · Gate 1 at week 3 (the 10-week PoC calendar of the 7 Oct brief is superseded)	Sponsor (go/no-go for build; validation location)
2 Solution Design	Weeks 4 to 5 · Google Cloud services confirmed, SAP method approved, review app design · Gate 2 at week 5	Weeks 4 to 5 · step list confirmed · Gate 2 at week 5	Technical lead (design approval)
3 Build	Weeks 6 to 10 · Gate 3 at week 10	Weeks 6 to 9 · Gate 3 at week 9	Sponsor (evaluation passes in Dev; build documentation and test cases delivered)
4 Test and Validation	Weeks 11 to 13 · Gate 4 at week 13	Weeks 10 to 12 · Gate 4 at week 12	Sponsor (release approval; UAT signed off, thresholds met, 5.4 findings closed, run model agreed)
5 Production Release	Weeks 14 to 17 · shadow then assisted, now per cohort · Gate 5 at week 17	Weeks 13 to 15 · shadow 1 week, assisted 2 weeks · Gate 5 at week 15	Sponsor (production acceptance; Phase 2 decision for Agent A)
6 Hyper Care and Handover	Weeks 18 to 21 · Knowledge Transition to the AMS partner · Gate 6 at week 22	Weeks 16 to 19 · Knowledge Transition to the AMS partner · Gate 6 at week 20	Run owner, with the AMS partner's sign-off (closure)

Both plans are fixed in SOW No. 07 (redline v2, 20 October 2026): both agents start in week 0 on 26 October 2026; Agent A reaches Gate 5 in week 17 and closes at Gate 6 in week 22, Agent B reaches Gate 5 in week 15 and closes in week 20. Stage 1's three weeks count from the day access and the historical sample are both in place (8.2(c)); Ingredion reviews

deliverables and gives gate decisions within 5 working days (4.2); scope and fees are not reopened at Gate 1 (8.2(c)), the scorecard thresholds are.

How this maps onto the repository

Implemented in the `Ingredion/` folder on 19 and 21 October (82 tests on stubs and a fake model; `python -m factory.evaluation prebill` returns the Kraft Heinz anchor at 4 of 4 with no false alarms). The shared runtime is one package; each agent is a set of thin step modules plus connector contracts whose real implementations are written against the interfaces Ingredion confirms in Stage 1.

```
Ingredion/
  factory/                      shared runtime; imports neither agent
    case.py                     Case, Flag, StepResult
    intake/                     state machines (A and B), one-case-once claims, pollers with markers
    guardrails/                 PatternGuard (Dev) · ModelArmorGuard (fails closed) · GuardChain
    model_gateway/              VertexStructuredExtractor (Gemini, response schema with JSON-mode fallback) ·
                                DocumentAIReader · Word, Excel and text readers · PromptRegistry
(prompts/*.vN.md) ·
                                GuardedExtractor · legacy Foundry client for the conversation API
    rules/                      closed reason-code catalogue (67 codes, routing, owners) · rule engine ·
composite confidence
    review/                     review queue (memory | Mongo) · /api/review router · decisions captured
    audit/                      append-only store (no update, no delete) · outcome markers
    evaluation/                 Order Intake and pre-bill scorecards · CLI
    observability/              OpenTelemetry → Cloud Trace · kill switch (env | memory | Mongo) · /api/ops
router
  order_intake_agent/           models · OrderIntakePolicy (mode primary | fallback) · connectors (Salesforce
queue and cases,
                                SAP reads, SAP order API, Graph drafts; protocols + stubs) · steps a1..a10 ·
                                pipeline (gate, exits to the manual queue, write-ahead submitting, lookup
before retry,
                                incompletion log) · case store · runtime
  prebill_agent/                canonical schema · normalise · four checks with verdicts · outcome labels and
owners per
                                redline v2 · connectors (read-only; Cloud Storage extract landing) · steps
b1..b8 ·
                                pipeline (recheck) · Kraft Heinz fixtures
  app/                          FastAPI service mounting the review and ops routers
  Ingredion_docs/redline-v2/    the three redline diagrams and the redlined SOW No. 07 this page follows
  tests/                        factory/ · order_intake/ · prebill/ · API
```

Still outside the code: real connectors (Stage 1 interfaces), the review page, single sign-on, child cases for multi-PO emails, the cross-cloud network path, and the backtest data, which Ingredion provides.

Sources: Statement of Work No. 07, Ingredion redline v2 (redacted), 20 Oct 2026 · Ingredion redline v2 diagrams, 20 Oct 2026 (Agent A proposed process and architecture on Google Cloud; Agent B future-state architecture and PO alignment flow) · Email Order Intake Agent, SOW scope sections Draft 3.0, 6 Oct 2026 · Email Order Intake Agent, SOW walkthrough deck Draft 3.0, 6 Oct 2026 · Email Order Intake Agent, responses to SOW clarification questions (added 19 Oct 2026) · Pre-bill Validation and Dispute Prevention Agent, SOW Draft 1.1 technical walkthrough, 10 Oct 2026 · Pre-Bill Validation business briefing, 2 Oct 2026 · discovery session transcript, 24 Sep 2026 · workflow sketch, 16 Oct 2026. Figures are quoted from the documents; proposals are marked as such and stand until each SOW's Gate 1 and Gate 2.